

Projet 3 – Parcours Business Intelligence Analyst – OpenClassrooms



Requêtez une base de données avec SQL

OBJECTIF DU PROJET

2

- ▶ Se familiariser avec l'écriture de requêtes SQL.
- ▶ Explorer deux tables : contrat et region.
- ▶ Croiser les deux pour répondre à des questions métiers simples.
- ▶ Produire un support expliquant la méthodologie (pas seulement les résultats).

SOURCE DES DONNÉES

3

► Tables utilisées :

- **contrat** : contrats d'assurance logement (surface, type de contrat, formule, valeur déclarée, prix de cotisation...) et **region** : informations géographiques (commune, département, région).
- Clé d'articulation : **contrat.Code_dep_code_commune = region.Code_dep_code_commune**

	Nom des colonnes	Type de données	Taille	Clé	Contrainte	Description
CONTRAT.CSV	Contrat_ID	INTEGER		Clé primaire	NOT NULL	Id unique pour les contrats
	No_voie	INTEGER			NOT NULL	Numéro dans la voie pour l'adresse du logement assuré
	B_T_Q	VARCHAR	1		NOT NULL	Indicateur éventuel de répétition pour l'adresse du logement assuré sur un caractère
	Type_de_voie	VARCHAR	10		NOT NULL	Type de voie pour l'adresse du logement assuré: rue, av (Avenue), rte (Route), ...
	Voie	VARCHAR	50		NOT NULL	Libellé de la voie pour l'adresse du logement assuré
	Code_dep_code_commune	VARCHAR	5	Clé secondaire	NOT NULL	Concaténation du code département et code commune pour avoir une clé unique
	Code_postal	VARCHAR	5		NOT NULL	Code postal pour l'adresse du logement assuré. Champ obligatoire.
	Surface	INTEGER			NOT NULL	Surface habitable du logement assuré, exprimée en mètres carrés. Champ obligatoire
	Type_local	VARCHAR	20		NOT NULL	Type de logement assuré (ex. Appartement, Maison, Studio...). Champ obligatoire
	Occupation	VARCHAR	15		NOT NULL	Statut d'occupation du logement assuré (ex. Locataire, Propriétaire). Champ obligatoire
	Type_contrat	VARCHAR	30		NOT NULL	Type de contrat d'assurance habitation (Résidence principale, Mise en location, ...). Champ obligatoire.
	Formule	VARCHAR	20		NOT NULL	Formule du contrat d'assurance habitation (ex. Classique, Intégral, Essentielle...). Champ obligatoire.
REGION.CSV	Valeur_declaree_biens	VARCHAR	15		NOT NULL	Tranche de valeur déclarée des biens mobiliers assurés (ex. 0-25000, 25000-50000...). Champ obligatoire.
	Prix_cotisation_mensuel	REAL			NOT NULL	Montant mensuel de la cotisation d'assurance (en euros). Champ obligatoire.
	Code_dep_code_commune	VARCHAR	10	Clé primaire	NOT NULL	Concaténation du code département et code commune pour avoir une clé unique
	reg_code	INTEGER			NOT NULL	Code numérique de la région administrative associée à la commune (référence INSEE). Champ obligatoire.
	reg_nom	VARCHAR	50		NOT NULL	Nom complet de la région administrative française (ex. Auvergne-Rhône-Alpes). Champ obligatoire.
	aca_nom	VARCHAR	30		NOT NULL	Nom de l'académie administrative associée à la commune (ex. Lyon, Aix-Marseille, Nice). Champ obligatoire.
	dep_nom	VARCHAR	30		NOT NULL	Nom du département administratif associé à la commune (ex. Ain, Gironde, Var, etc.). Champ obligatoire.
	com_nom_maj_court	VARCHAR	50		NOT NULL	Nom de la commune en majuscules et sous forme abrégée (ex. AMBERIEU EN BUGEY). Champ obligatoire, utilisé pour les jointures et la normalisation des adresses.
	dep_code	INTEGER			NOT NULL	Code du département correspondant à la commune (ex. 1 pour Ain, 27 pour Eure). Clé étrangère vers la table des départements. Champ obligatoire.
	dep_nom_num	VARCHAR	40		NOT NULL	Nom complet du département suivi de son code entre parenthèses (ex. Ain (01)).

SCHEMA RELATIONNEL

4

contrat	
123 Contrat_ID	INTEGER NOT NULL
AZ B_T_Q	VARCHAR
AZ Code_dep_code_commune	VARCHAR NOT NULL
AZ Code_postal	VARCHAR NOT NULL
AZ Formule	VARCHAR NOT NULL
123 No_voie	INTEGER NOT NULL
AZ Occupation	VARCHAR NOT NULL
123 Prix_cotisation_mensuel	REAL NOT NULL
123 Surface	INTEGER NOT NULL
AZ Type_contrat	VARCHAR NOT NULL
AZ Type_de_voie	VARCHAR NOT NULL
AZ Type_local	VARCHAR NOT NULL
AZ Valeur_declaree_biens	VARCHAR NOT NULL
AZ Voie	VARCHAR NOT NULL

region	
AZ Code_dep_code_commune	VARCHAR NOT NULL
AZ aca_nom	VARCHAR NOT NULL
AZ com_nom_maj_court	VARCHAR NOT NULL
123 dep_code	INTEGER NOT NULL
AZ dep_nom	VARCHAR NOT NULL
AZ dep_nom_num	VARCHAR NOT NULL
123 reg_code	INTEGER NOT NULL
AZ reg_nom	VARCHAR NOT NULL

- Un contrat est rattaché à 1 commune/département/région → via Code_dep_code_commune.
- On peut donc on peut **lister**, **compter**, **moyenner** par territoire.
- Table contrat (PK = Contrat_ID et **FK = Code_dep_code_commune**).
- Table region (**PK = Code_dep_code_commune**).
- Génération du code SQL de création des tables dans Dbeaver, client SQL Universel avec utilisation de SQLite, SGBD.
- Nombre de lignes chargées dans la base (Cf Document technique) :
 - Table contrat = 30335.
 - Table region = 38916.

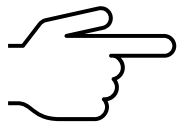
1. Identifier la question métier (ex. “contrats à Caen”, “prix moyen par région”).
2. Identifier les **tables** nécessaires (contrat seul ou contrat + region ?).
3. Écrire la jointure minimale.
4. Ajouter **filtres** (WHERE) et **agrégations** (GROUP BY).
5. Vérifier avec des requêtes de contrôle (tri, total).

- R1 : contrats à Caen

```
SELECT  
    c.Contrat_ID,  
    c.Surface,  
    r.com_nom_maj_court AS commune  
FROM contrat c  
JOIN region r  
    ON c.Code_dep_code_commune = r.Code_dep_code_commune  
WHERE r.com_nom_maj_court = 'CAEN';
```

- R3 : liste des régions

```
SELECT DISTINCT reg_nom FROM region;
```



Je sais lire les clés, faire une jointure interne, filtrer.

RESULTATS REQUETES 1 ET 3

7

Contrats à Caen

	123 Contrat_ID ▼	123 Surface ▼	AZ commune ▼
1	103 791	35	CAEN
2	103 792	99	CAEN
3	103 793	40	CAEN
4	103 794	20	CAEN

Liste des régions

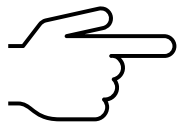
	AZ Régions de France ▼
1	Auvergne-Rhône-Alpes
2	Bourgogne-Franche-Comté
3	Bretagne
4	Centre-Val de Loire
5	Collectivités d'outre-mer
6	Corse
7	Grand Est
8	Guadeloupe
9	Guyane
10	Hauts-de-France
11	Ile-de-France
12	La Réunion
13	Martinique
14	Mayotte
15	Normandie
16	Nouvelle-Aquitaine
17	Occitanie
18	Pays de la Loire
19	Provence-Alpes-Côte d'Azur

- R11 : Liste des communes >= 150 contrats

```
SELECT
    r.com_nom_maj_court AS commune,
    COUNT(c.Contrat_ID) AS nb_contrats
FROM contrat c
JOIN region r
    ON c.Code_dep_code_commune = r.Code_dep_code_commune
GROUP BY r.com_nom_maj_court
HAVING COUNT(c.Contrat_ID) >= 150
ORDER BY nb_contrats DESC;
```

- R5 : Prix moyen de
cotisation
mensuelle

```
SELECT ROUND(AVG(Prix_cotisation_mensuel), 2) AS prix_moyen_cotisation_mensuelle
FROM contrat
WHERE Prix_cotisation_mensuel IS NOT NULL;
```



J'ai utilisé GROUP BY + HAVING quand il fallait filtrer sur l'agrégat.

RESULTATS / REQUÊTE 5 ET 11

9

Prix moyen de la cotisation mensuelle

	123 prix_moyen_cotisation_mensuelle ▼
1	19,33

Liste des communes >= 150 contrats

	A-Z commune ▼	123 nb_contrats ▼
1	PARIS 18	515
2	PARIS 17	468
3	PARIS 15	407
4	PARIS 16	394
5	NICE	387
6	PARIS 11	381
7	PARIS 20	302
8	BORDEAUX	302
9	NANTES	291
10	PARIS 19	266
11	PARIS 10	263
12	PARIS 12	252
13	PARIS 14	222
14	GRENOBLE	220
15	PARIS 9	204
16	TOULOUSE	187
17	TOULON	170
18	COURBEVOIE	163
19	LILLE	161
20	PARIS 3	159

Requête 9 : Surface moyenne des contrats à Paris

- La donnée "Paris" était éclatée : PARIS, PARIS 1, ..., PARIS 20.
- On a normalisé les espaces avec REPLACE(...).
- On a pu ensuite calculer une **surface moyenne intégrant tous les arrondissements de Paris**

```
WITH paris_norm AS (  
  SELECT  
    c.Surface,  
    REPLACE(r.com_nom_maj_court, ' ', '') AS commune  
  FROM contrat c  
  JOIN region r  
    ON c.Code_dep_code_commune = r.Code_dep_code_commune  
)  
SELECT  
  ROUND(AVG(CAST(Surface AS REAL)), 2) AS surface_moyenne_paris  
FROM paris_norm  
WHERE  
  commune = 'PARIS'  
OR commune GLOB 'PARIS [1-9]'  
OR commune GLOB 'PARIS 1[0-9]'  
OR commune = 'PARIS 20';
```

	123 surface_moyenne_paris ▼
1	51,77

- Bien identifier les **clés de jointure** avant d'écrire la requête.
- Utiliser GROUP BY + HAVING pour les filtres sur des agrégations.
- Tester les résultats intermédiaires (ex. SELECT DISTINCT commune ...) avant d'écrire la requête finale.
- Documenter chaque requête dans un fichier séparé.

- La base est exploitable pour faire des analyses géographiques simples.
- On sait maintenant : lister, compter, moyenner, classer, joindre.
- Étape suivante possible : vues, nettoyage dans Power Query, ou exposer ça dans un outil de dataviz.